

AI + Gamedev

Making games with AI tools. How to start, and how not to.

Darryl Wright · Punk Science Studios Inc · Bossa Games · Halifax, Nova Scotia · 2026

Before we start

What I'm going to skip

- **What an LLM is** and how it works.
- **Chatbots** and the prompt-and-response loop.
- **Third-party publishing.**

Forty-five minutes + fifteen minutes for questions

What we're going to cover

- The (evolving) game development cycle
- Defining terms
- What's a harness, and how much does it matter
- The model landscape
- Agents and skills
- Workflow, and choosing a harness
- Good planning
- Good architecture
- Platform, engine and art
- The state of the art
- The back end
- Prompt and context tips
- Sovereignty, and why it's a thing now
- Questions

You could one-shot a game, but game developers care more. AI accelerates a well-honed discipline.

The (Evolving) Game Development Cycle



Same stages. Every one of them faster. Alone, if you have to.

The same ring, on fast-forward



■ Every stage still happens.

Ideas, plans, code, art, builds, ops. None of them go away. AI compresses all five, and Production most of all: that is where the code gets written and rewritten, in code and in art.

■ What stays human, in every stage.

Taste. The design call. Knowing a build is good enough. Art direction. The publisher relationship. The decision to ship.

Gold is dead. Ship-and-forget is dead. The ring turns in days now. One person can turn it. I have. That is what this talk is about.

Defining Terms



Defining Our Terms

Front end

Game

SDK

Client

API

Server

Back end

What's a Harness, and How Much Does It Matter



The model

A brain full of inert knowledge, with no action

No legs, no direction.

The harness is the tool for intention and focus. It gives the horse the two things it doesn't have on its own: reach, and restraint.



A chat window is a harness with none of this

The harness adds two things

Reach.

Tools. A shell. The file system. The loop that lets the model do something, look at what happened, and do the next thing.

Let's be clear about this — a bare model can't open a file, run a build, or check its own work. It isn't wild. It's inert.

Restraint.

Permissions. Context limits. Hooks. Skills and agents that say how work gets done around here.

The harness and everything you put in it are yours. The model is rented. Remember that one; it comes back later.

The same prompt, twice

"Make me a space taxi game"

Chat window

You > Make me a space taxi game in three.js

Assistant > Sure! Here's a complete space taxi game. Copy this into an HTML file:

```
<script type="module">
import * as THREE from 'three';
const scene = new THREE.Scene();
const cab = new THREE.Mesh(
  new THREE.BoxGeometry(1, .5, 2),
  new THREE.MeshStandardMaterial());
... 412 more lines ...
</script>
```

Let me know if you'd like any changes!

No file. No run. No test. No memory of it tomorrow.

Terminal · Claude Code

> Make me a space taxi game in three.js

```
read AGENTS.md · docs/GDD.md
glob src/**/*.js (14 files)
write src/flight/spaceFlight.js
write tests/flight-modes.test.js
bash npm test
    27 passed, 0 failed
bash npm run dev
shot http://127.0.0.1:5173 → frame.png
    The cab drifts up on entry.
    Trimming vertical thrust at the boundary...
bash npm test    28 passed
git commit -m "Space flight mode - refs #4"
```

A file, a test, a screenshot, a commit.

The model answered. Nothing happened.

The harness did the work. Reconstructed from a real O'Neill session.

The Model Landscape



The names go stale in months. The shape of the thing does not.

How many is a lot?

Where	Count	Rate	Pulled
OpenRouter, models listed	446	39 added in the last four weeks. About 10 a week.	2026-09-20
Hugging Face, public models	3,080,517	About 3,000 a day. They passed three million on August 18, 2026.	2026-09-20
OpenRouter, tokens routed	20 trillion a week	April 2026 figure, secondary source	2026-09-20

Sources: openrouter.ai/api/v1/models (counted), huggingface.co/models, the Hugging Face "three million models" post. Nobody can evaluate this many. Nobody is going to try.

So the useful question isn't "which model." It's "which kind, and where does it run?"

By kind, then by where it comes from

The categories outlive the model names

Tool-calling

Can drive a harness: read, write, run, check. The only kind that matters for an agent. Most frontier and open-weights models now qualify.

Coding

Tuned on code and repos. Claude, GPT and Gemini at the top; Qwen Coder, DeepSeek and North Mini Code as open weights.

Image and 3D

Flux, Imagen and GPT Image for 2D. Meshy, Tripo, Rodin and Hunyuan3D for meshes. A different pipeline, and a different conversation, later.

Frontier, US

Anthropic's Claude, OpenAI's GPT, Google's Gemini, Meta's Muse Spark. Closed weights, hosted, the strongest planners.

Open weights, China

Qwen 3.8, DeepSeek V4, Kimi K3, GLM-5.3. Run them anywhere, including your own box. Read the licence.

Canadian

Cohere's Command A+ and Command A, out of Toronto. North Mini Code: 30B, Apache-2.0, runs locally. It exists, and it's a lot closer to home.

Model names as of 2026-09-20. The six categories will still be here next year. Some of the model names won't.

One endpoint, every model

OpenRouter, and the "auto" model

The on-ramp.

One API key instead of accounts at five labs. Every model on the last slide, behind one URL.

openrouter/auto picks the model for each prompt, "powered by the wisdom of the market" — which is to say, whatever the crowd is spending on this week, by task type.

It's the fastest way to experiment. Use it tonight.

The catch.

You no longer know which model touched your code.

That matters for **cost** (a hard problem routed to a frontier model bills like one), for **consistency** (your agents and skills are tuned to a model's habits), and for **sovereignty** (where did your repo just go?).

Experiment on auto. Build on a pinned model.



openrouter.ai

**Frontier to plan.
Cheap to grind.
Local when you can afford the hardware,
or **need privacy**.**

Agents and Skills



The leverage is in what you write down, not in what you type

A recipe and a specialist

Skill. A reusable set of instructions the harness loads when a task matches.

```
skills/ps-project-init/SKILL.md
---
name: ps-project-init
description: Initialize a project workspace.
  Scan for existing setup and docs, write
  a project AGENTS.md and a CLAUDE.md
  pointer, init git, make the first commit.
  Use when the user says "init this repo"...
---
## Steps
1. Scan the tree for package.json, ...
2. Draft AGENTS.md with the sections ...
```

Agent. A persona with its own rules, tools and context that you hand a job to.

```
agents/art.md
---
name: art
description: Art, a senior art director.
  Takes a stated vision and constrains it
  into an enforceable art bible...
model: opus
color: green
---
You are Art, a senior Art Director...
Process intake → interrogation
  (five questions, max) → the reduction
  → systems → art bible → enforcement.
```

Both are Markdown with a frontmatter block. No code. That's the whole trick, and it has to be said: it's a good one.

This deck was designed by the Art agent. This morning.

What an agent hands back

```
> art: Produce the art bible for a conference slide deck. No user available: ask your five questions in the document, answer them yourself, proceed.
```

```
Art > Art Bible written to docs/art-bible.md.
```

The direction in one line. Near-black clinical slide system, cream type, one acid lime, and lime only ever marks where a human is still required.

Decisions worth flagging to you.

- Two brand greens demoted on contrast grounds: dark olive measures 2.11:1 on black, invisible on a weak projector.
- Univers Light Condensed rejected: light condensed dies at distance. Archivo instead.
- The horse has no legs. The torso just stops. Nobody in the picture acknowledges it.
- The live-ops segment ends in a 24° gap. The ring never closes. Do not tidy.

```
direction_contract:
  palette: { ground: "#111111", text_primary: "#F0EDE4", signal: "#C8DF00", border: "#7A8800"
}
  contrast: { body_min: 7.0, projector: "assume washed out; ratios are best case" }
  motion: { allowed: 2, slide_advance: 120ms, build_reveal: 180ms }
```

A YAML block other agents parse without reading the prose. The build agent that wrote this HTML read the contract, not the essay. I overruled exactly one thing: it wanted the whole deck in lowercase.

Start with other people's. Then write your own.

Where to get them



This talk, and my
cleared skills and
agents



Matt Pocock's skills
grill-me, tdd, handoff



devdavr / unity-ai-
workflow
brainstorm, the uw-
family



donchitos /
claude-code-game-
studios



anthropics / skills
The official set

Workflow, and Choosing a Harness



Experiment. Find the one that suits you. Don't take my word for it.

Six harnesses, one multiplexer

Harness	Where it lives	Models	Skills · agents	Cost	Local models
Claude Code	Terminal, IDE, desktop, web	Anthropic only	Skills, subagents, hooks, MCP	\$20 to \$200 a month, or API	No
Codex CLI	Terminal, IDE	OpenAI	Skills, MCP, plugins	ChatGPT plan or API	Via Ollama
OpenCode	Terminal	75+ providers, OpenRouter	Agents, skills, MCP	Free, open source	Yes: Ollama, LM Studio
Muse Code	Terminal	Meta's Muse Spark only	Subagents, worktrees	\$5 to \$50 a month	No
Pi	Terminal	15+ providers, OpenRouter	Extensions, skills	Free, pay your own tokens	Yes: Ollama
Buzz	Desktop app	Drives Claude Code, Codex, goose	Humans and agents as peers	Free, self-hostable	Whatever it drives
herdr	Terminal	Not a harness. A multiplexer: every agent pane at a glance, working, blocked or idle		Free, Apache-2.0	n/a

Pulled 2026-09-20. Lime is my own split: Claude Code when I want reliability and renewability, Pi with OpenRouter for the rest. Fair warning — costs mount fast on hard problems.

Why I keep ending up back in a terminal

Close to the input, close to the output

Terminals are fast.

They hold you right next to the prompt, the output, and the result of every tool call. Cut and paste between five sessions in herdr. SSH into another box. Git is right there.

Sure, the terminal costs you a day of discomfort. That day is the price of everything after it.

Desktops hide and simplify.

And somehow end up more complicated. In Claude Desktop I don't know where I'm supposed to do what, and when it works on a codebase I can't tell you where my changes actually landed. Same with Buzz.

A chat window with a repo attached is still a chat window.

My daily loop: plan in the vault, grill the plan, init the folder, implement in small commits, review, repeat.

Good Planning



Begin with a folder. Keep everything in it.

Brainstorm, grill, init

Brainstorm

ps-brainstorm · unity-ai-workflow's brainstorm
A one-page GDD: pillars, core loop,
Gherkin scenarios, a feedback matrix

Grill

Matt Pocock's grill-me
The agent interviews you until every
open question in the plan is closed

Init

ps-project-init
AGENTS.md, CLAUDE.md, git, first commit.
The agent's context lives with the code

Then, and only then, code.

The plan is a document the agent reads on every task — not a conversation that scrolled away last Tuesday.
One folder, one repo: plan, context, code, tests, art bible. Portable to any harness, any model, any machine.

Snail's Pace, a real project, as of this morning

What a finished plan looks like

AGENTS.md

Project-specific context for AI coding agents.

Read these first – do not duplicate them

- README.md – pillars, build, the browser-first rule
- docs/SNAILS_PACE_GDD.md – Core Loop GDD. Mechanics as Gherkin: this is the build contract.
- docs/ART_BIBLE.md – "Toy Theatre": the value ladder, palette tokens, the art lint. LAW = machine-checked in CI; RULE = needs a written reason in the PR.

Commands

- npm ci · npm run dev · npm test
- npm run lint:art && npm run lint:vocab
- npm run build → dist/ · npm run android:apk

AGENTS.md · written by ps-project-init, commit 44c2500

SNAILS_PACE_GDD.md

Pillars

1. The game plays while you're away. Steps set velocity, not position.
2. Slowness is the joke. Duration is the punchline.
3. Identity over power. Cosmetics only.
4. Every screen is a screenshot.

Core Loop

WALK (real world input, passive)

→ steps sync → slime tank fills → crawl rate rises

CHECK IN (3-6×/day, 20-60s sessions)

→ macro camera shows ground gained since last look

→ one fresh incident card (the joke)

→ rival gap delta (the stakes)

GDD v0.9 · pillars, core loop, Gherkin acceptance, feedback matrix

Good Architecture



**Even if the AI wrote it,
I wrote it.**

Architecture is for the two parties who still have to work in the code: the agent, and the team

When is a codebase ready for an agent?

- **When it's indexed.** The codebase-memory skill builds a knowledge graph of the functions, classes and calls, so "who calls this?" is a query the agent runs, not a hundred files it reads.
- **When it builds from one command.** The agent runs the build exactly the way you do. No clicking through an editor.
- **When it tests from one command.** The tests sit beside the modules and map back to the scenarios in the plan.
- **When the plan is in the repo.** AGENTS.md, the GDD, the art bible. The agent reads them at the start of every task.
- **When the agent can see what it did.** A dev server, a screenshot tool, a render script, a log it can search.

One more thing. AI is not the best tool for walking a tree, parsing a file, or rote replacement such as linting and renaming. It can do those things. Should it? Use the linter, and save the model for the parts that need judgement.

Platform, Engine and Art



What the agent can touch, and what it can't

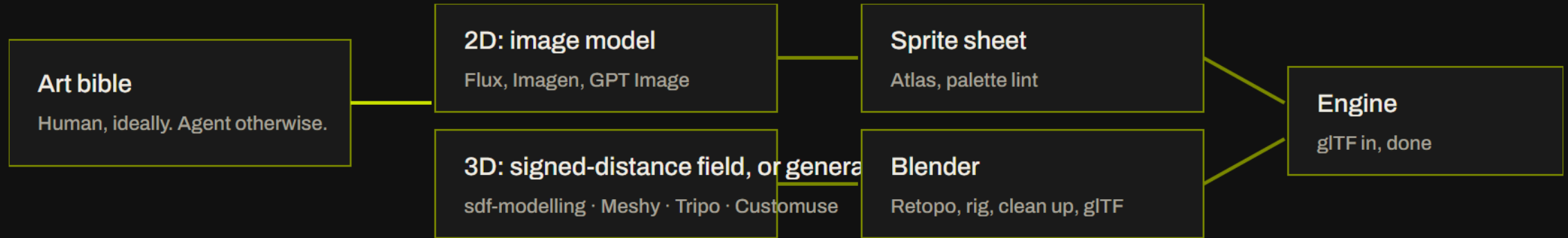
Engines, from the agent's side

Engine	Text and diffable	Headless build and run	Editor access	Art in	Ships to
Web: Phaser, three.js	All of it	npm run build, Playwright	None needed	glTF, sprite sheets, atlases	Browser; Android and iOS via Capacitor; Steam via Electron
Unity	C# yes; scenes and prefabs are YAML, and fragile	Batchmode builds, test runner	MCP for Unity, GladeKit	FBX, glTF, Blender direct, sprite atlas	Everything, with a licence
Godot	GDScript and .tscn are text	Headless export	Godot MCP, CLI	glTF native, Blender direct, sprites	Desktop, mobile, web; console via partners
Unreal	C++ yes; Blueprints and assets are binary	Unreal Automation Tool	Python API, thin MCP	FBX, glTF, USD	Everything, at scale

Lime is the fastest first game for a newcomer: everything is text and it runs in a browser tonight. Unity is where the professionals in this room already live, and it works well with MCP. Blueprints and binary assets fight the agent every step of the way.

Where the assets come from

The AI art pipeline



This is still the weak point of the whole process.

Humans for decisions, direction, taste and concepts.

AI tooling for the repetitive and the many: variants, sheets, filler, retakes.

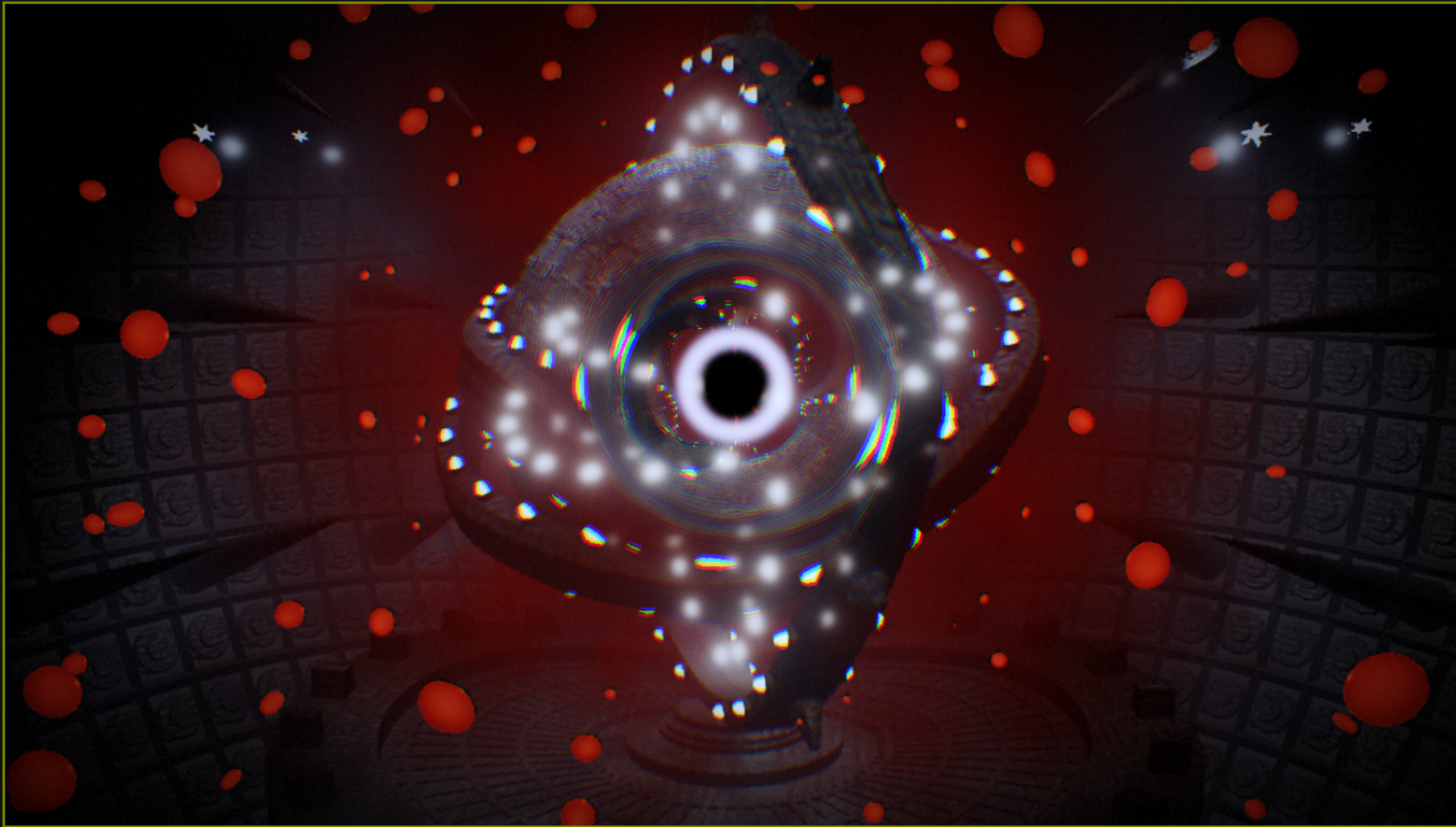
The art bible and a human eye are what keep it from looking like AI slop. Nothing else does.

The State of the Art



An in-house tech demo · the live version is at the link

Gravity Drive



[punkscience.ca/
gravity-drive/](https://punkscience.ca/gravity-drive/)

The drive from *Event Horizon* and its second containment chamber. Built twice: as a parametric Blender scene driven through Blender MCP, and as this single-file WebGL demo.

How it was made: the two parts that matter

The agent sees its own frames

1. It can look.

```
# Stills of one sequence state, for judging a change
node tools/shot.mjs --state 2 --at 2,4,6,8 --out
renders/breach

# The reel, against a running server
node tools/capture.mjs --out renders/reel-60s.mp4 \
  --w 1080 --h 1920 --fps 30 --seconds 60

# The Blender half, through the MCP addon
exec(open("blender/assemble_scene.py").read())
# Idempotent: purges and rebuilds the whole scene
```

Render tools the agent wrote for itself. Every change to the look gets judged from a frame, not from the code.

2. One issue per task, unattended.

AGENTS.md · Workflow

Nothing is worked on that is not an issue.

1. gh issue list; pick the issue
2. Label: status: in progress
3. Branch off main: fix/<slug> or feature/<slug>
4. Do the work, commit there
5. Merge to main, push, close the issue

git log

New Feature: access corridor, character controller

New Feature: thud on the ring lock; charge loop
winds down through DECAY

Fixes the charge loop level for the replaced charge.wav

The loop the agent runs without a human watching every step. The log is the audit trail.

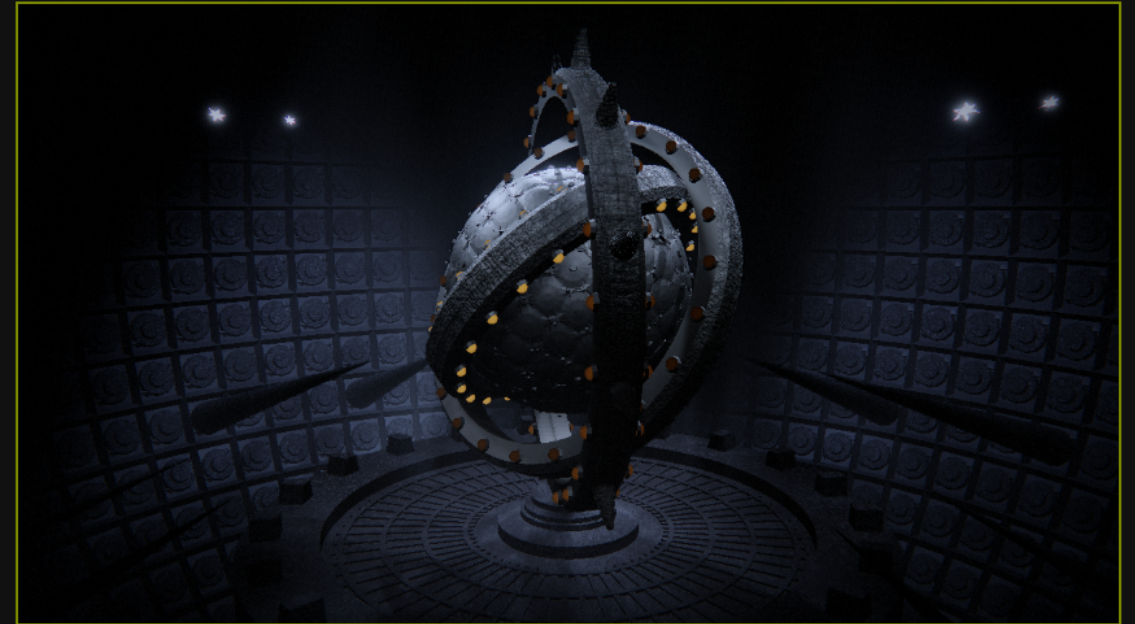
The same tools, the same week

What worked, and what did not

Blender + Blender MCP · the monster

Six attempts at an organic creature.
Sculpting is not a thing an agent can do.
It can place primitives and hope.

No usable render survived.



The monster: Blender + Blender MCP. No usable result.

The drive: parametric, numeric, judged from its own frames. Shipped.

Geometry you can describe with numbers, the agent can build. Geometry you have to sculpt, it can't. That's the art verdict from two slides ago — this time with evidence.

The Back End

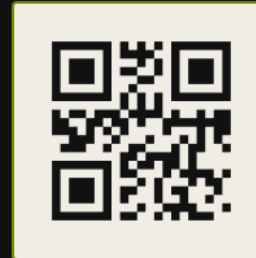


Strictly speaking, a game only needs a game

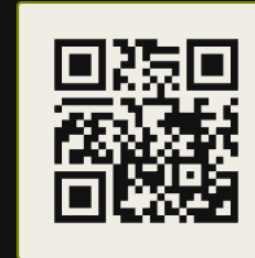
"I can't do server stuff" is over

- **A box** that's always on, or a tunnel to one. A \$2 VM, a Pi, a Halifax host.
- **An API** the agent scaffolds: REST or WebSocket, same session as the client.
- **CI** that builds every push. Snail's Pace publishes an Android build on every merge.
- **A kill switch.** Because live ops.
- **A plan for success.** Scale is a good problem to have.

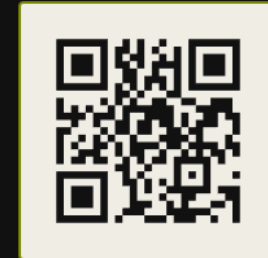
Snail's Pace has no server of its own. The race syncs between two phones over Nostr relays, and the invite is a link the studio site serves. A protocol did the back end. That's not a trick — that's the point of a protocol.



fly.io



websavers.ca
Halifax, since
2004



nostr-book.org

Prompt and Context Tips



The slide you photograph

Seven things I actually do

- **Write** the plan to a file and point the agent at it. Never re-explain it in chat.
- **Name** the layer you mean. "The HUD," not "the score thing."
- **Grill** before building. The agent asks, you answer, the plan closes.
- **Commit** small. One issue per task, one branch per issue.
- **Keep** AGENTS.md current. It's the only memory that survives the session.
- **Let** the agent see its own output: tests, renders, screenshots, logs.
- **Pin** a model. Experiment on auto, build on one you know.

And three more, about context itself

Knowledge that doesn't leak

- **More** context is always better. More of the **right** context. Read the plan, the GDD, the art bible, the last commit.
- **Write** skills that update themselves. When the agent works something out the hard way, the last step is: write it to a skill. That knowledge is yours now, not the session's.
- **Format** agent-facing text as Simplified Technical English. ASD-STE100 is the standard aerospace uses so a maintenance instruction can't be misread. An agent reading another agent is in exactly the same position.



asd-ste100 skill
danyuchn, MIT

The counterweight to "more context is better"

Keep your eye on the context window

- **It fills up faster than you expect.** Every file the agent reads, every tool result, every screenshot goes into the window and stays there.
- **Watch the status line.** The harness shows you how much of the window is left. Most people don't look at it until it has run out.
- **Compaction loses things.** When the harness summarizes the conversation to make room, the specific detail you cared about is usually what gets dropped.
- **Hand off instead of pushing on.** Write the current state to a file, start a fresh session, and point the new session at that file.
- **Scrollback is not context.** The plan in the repo is context. The argument you had with the agent an hour ago is not, and it will not survive.

**"Make me a game"
is not a prompt.
It's a **wish**.**

Sovereignty, and Why It's a Thing Now



Three risks and one hedge

Whose model is it, anyway?

- **Your IP crosses a border** on every API call. In a trade war, a model can become an illegal export overnight. There is no local copy of Claude.
- **The provider can change the deal.** Terms, pricing, what they train on. Meta's contributor tier trains on your sessions. Do you want to train their models with your ideas?
- **The model can be retired.** The one your skills were tuned to. And DeepSeek's API sends your repo to a company that answers to Beijing. Are you fine with that?
- **The hedge:** keep the plan, the skills and the context in **your own repo**, so any model can pick them up. Know how to run an open-weights model locally, even if you rarely do.

**Where the model runs
decides who sees **your work**.**

Start Tonight



Planned, grilled, built, tested and art-directed with this workflow

O'Neill: Orbital Taxi



A three.js prototype in the spirit of *Space Taxi* and *Lunar Lander*, inside a procedurally modelled O'Neill cylinder. You start outside with Earth behind you, fly the arrival corridor, and land on the pads inside.

Planned, grilled, built, tested and art-directed exactly the way this talk describes. A few weeks of evenings, and yes, a few burnt steaks.

Keyboard: W/S thrust, arrows aim, Space/Ctrl trim. A controller works too.

Every link from the talk, in one place

Links



This talk, skills,
agents
GitHub



Gravity Drive, live
punkscience.ca



herdr
Agent multiplexer



OpenRouter
One key, every
model



pi.dev
The minimal
harness



OpenCode
Open source



agent-skills
APT, Chocolatey,
wiki



punkscience.ca
The studio

Questions.

Darryl Wright
punk.science
Halifax, Nova Scotia



The talk, the skills,
the agents